Extended reading — Week 5: Function Approximation and DQN

STUDY-RL

October 20, 2026

Read with a question in mind

Start with the week's teaching guide and its small worked example. Then use these readings to connect the algorithm to its assumptions and evidence. The explanations and investigations below are course commentary, not quotations. Read the source before reusing a theorem or a reported result. For each paper, record the objective, observation model, claimed result, conditions and one condition your local experiment does not reproduce.

1. Mnih et al. (2015): Human-level control through deep reinforcement learning

Primary source or book record: https://doi.org/10.1038/nature14236.

What to learn. Combines a neural action-value function with replay and a target network in a successful Atari system.

What the result requires. The local small-environment implementation teaches the update structure; it is not an exact reproduction of the published image preprocessing, architecture, compute budget or scores. Deep RL did not begin with this single paper.

Notebook investigation. Use `exercise_1_gather_q` and `exercise_2_dqn_target`. Check the batch dimension and detached bootstrap. A valid target changes only when its inputs or target parameters change.

2. Sutton and Barto (2018): Reinforcement Learning, Chapter 11

Primary source or book record: http://incompleteideas.net/book/the-book-2nd.html.

What to learn. Explains why function approximation, bootstrapping and off-policy sampling can interact badly.

What the result requires. The deadly triad identifies a failure possibility, not a theorem that every such system diverges. Stable training in one experiment does not remove the theoretical issue.

Notebook investigation. In `exercise_7_baird_divergence`, inspect parameter norm as well as prediction error. A decreasing error on frequently sampled states can hide instability elsewhere.

3. Baird (1995): Residual Algorithms

Primary source or book record: https://doi.org/10.1016/B978-1-55860-377-6.50013-X.

What to learn. Uses counterexamples and residual-gradient constructions to examine value learning with function approximation.

What the result requires. A counterexample refutes a universal guarantee; it does not predict how often a practical deep algorithm fails. The fixed-point semi-gradient and a gradient of Bellman residual are different updates.

Notebook investigation. Write the derivative of a squared TD residual both with and without differentiating through the successor prediction. Explain why the two vector fields need not share behaviour.

4. van Hasselt et al. (2018): Deep Reinforcement Learning and the Deadly Triad

Primary source or book record: https://arxiv.org/abs/1812.02648.

What to learn. Empirically studies instability in deep Q-learning systems with replay and variations in algorithm design.

What the result requires. Observed value growth, divergence and low return are related but different outcomes. Do not assign every loss spike to one cause without inspecting rewards, targets, gradients and data.

Notebook investigation. Log target magnitude, online Q magnitude and gradient norm during one controlled change. State which measurement supports a stability claim and which would falsify it.

5. Farebrother, Machado and Bowling (2018): Generalization and Regularization in DQN

Primary source or book record: https://arxiv.org/abs/1810.00123.

What to learn. Studies transfer across Atari modes and evaluates regularisation and representation reuse.

What the result requires. Training-environment return is not the same as generalisation to a changed environment. The paper does not establish that every regulariser improves every DQN task.

Notebook investigation. In the workbook investigation, separate training and evaluation seeds or dynamics. Keep a fixed evaluation protocol and report uncertainty across runs, not only the best checkpoint.

Synthesis: evidence you should be able to show

1. Reproduce one numerical calculation before running a long experiment.
2. Name the precise objective, sampling distribution and terminal convention.
3. Compare the local implementation with one source: list a shared mechanism and one simplification that prevents calling it a full reproduction.
4. Separate an algebraic identity, an assumption-dependent theorem and an empirical result. A plot alone does not prove a theorem.
5. Save the notebook with seed, parameters, axes, denominators and a short explanation of an unexpected result. Keep evaluation data separate from tuning.

Return to the teaching guide's notebook investigation for the local function names, expected checks and interpretation questions. Source code and notebooks are available together in the course repository; the web reader is the entry point and the downloaded notebook is the execution workspace.